

Day 1 - Thinking in Algorithms

DuplicateCheck.java

```
import java.util.HashSet;
```

```
public class DuplicateCheck {
```

// Compare every pair : $\frac{n \times (n-1)}{2}$ Comparison $\rightarrow O(n^2)$
 $\rightarrow$ int type array as input

```
static boolean hasDuplicateNested(int[] a) {
```

```
    for (int i = 0; i < a.length; i++) { // length = 10 - 0  $\rightarrow$  9
```

```
        for (int j = i + 1; j < a.length; j++) { i = 0
```

```
            if (a[i] == a[j]) { j = 1
```

```
                return true;
```

```
            }
```

```
        return false;
```

```
    } }
```

Ex

0	1	2	3	4	5
1	2	3	4	4	2

 i j j j j j

return true

$\rightarrow$ int type array

```
static boolean hasDuplicateSet(int[] a) {
```

Collection $\leftarrow$ Set<Integer> seen = new HashSet<>(); a = [1, 2, 4, 5]

```
for (int x : a) - For each array
```

```
    if (!seen.add(x)) return true;
```

```
    return false;
```

```
}
```

```
public static void main (String[] args) {
```

```
    for (int n = 10; n <= 200; n *= 2) {
```

array $\leftarrow$ int[] a = new Random(7).ints(n, 0, Integer.MAX_VALUE)

```
        .distinct().toArray();
```

```
        long t = System.nanoTime(); // Time to execute program
```

```
        boolean r1 = hasDuplicateNested(a);
```

```
        long nested = (System.nanoTime() - t) / 1000000;
```

```

t = System.nanoTime();
boolean x2 = hasDuplicateSet(a);
long set = (System.nanoTime() - t) / 1000000;
Sout("n=%d, 8d loop:%6d ms HashSet %6d ms", a.length, nested, set, x1, x2);

```

Patterns

Pattern 1 - single loop: $O(n)$ i=0 n=5
 for (int i=0; i<n; i++) count++; 0 1 2 3 4

Pattern 2 - nested, independent: $n \times n = O(n^2)$
 for (int i=0; i<n; i++) n=3

for (int j=0; j<n; j++) i=0 1 2 Count=0
 count++; j=0 0 0 Count=9
1 1 1
2 2 2

Pattern 3 - Nested, inner depends on outer: $0+1+\dots+(n-1) = \frac{n(n-1)}{2} = O(n^2)$

for (int i=0; i<n; i++) n=3
 for (int j=0; j<i; j++) i=0 1 2
 count++; j=0 0 Count=0 → 3
1

Pattern 4 - Halving (or doubling): $O(\log n)$

for (int i=n; i>1; i/=2) count++;
n=5 → 3 Count=0+1+1=2
i=5 5 4 3 2

Pattern 5 - linear outer x logarithmic inner : $O(n \log n)$

for (int i = 0; i < n; i++)

for (int j = 1; j < n; j *= 2)

count++;

count = 0 + 1 + 1 + 1

= 3 + 3 = 3 * 5 = 15

n = 5

i = 0, 1, 2, 3, 4 = 5

j = 1, 2, 3, 4

Pattern 6 - Two loops one after the other $O(n) + O(n) = O(n)$

for (int i = 0; i < n; i++) Count++;

for (int i = 0; i < n; i++) Count++;

Pattern 7 - loop until $i * i > n$: $O(\sqrt{n})$

for (int i = 1; i * i <= n; i++) Count++

n = 5

i = 1, 2, 3

i = 1, 4, 8

1 + 1 = 2 → Count

Pattern 8 - fixed number of iterations : $O(1)$ no matter how big n is

for (int i = 0; i < 100; i++) Count++;

Loop Complexity, java

public class LoopComplexity {

static long single (int n) { $O(n)$

long c = 0;

for (int i = 0; i < n; i++) c++;

return c;

}

n = 5

i = 0 1 2 3 4

c = 5

static long nested (int n) { $O(n^2)$

long c = 0;

for (int i = 0; i < n; i++) {

for (int j = 0; j < n; j++) {

c++; } } return c;

c = 25

n = 5

i = 0 1 2 3 4

j = 0 0 0 0 0

1 1 1 1 1

2 2 2 2 2

3 3 3 3 3

4 4 4 4 4

static long triangle (int n) { $\frac{n(n-1)}{2} = O(n^2)$

long c = 0;

for (int i = 0; i < n; i++) {

for (int j = 0; j < i; j++) c++; }

return c;

c = 10

n = 5

i = 0 1 2 3 4

j = 0 0 0 0

1 1 1

2 2

3

static long nLogN (int n) {

long c = 0;

for (int i = n; i > 1; i /= 2)

for (int j = 1; j < n; j *= 2) c++; }

return c;

$O(n \log n)$

n = 5

i = 0 1 2 3 4

j = 1 1 1 1 1

2 2 2 2 2

4 4 4 4 4

6 6

c = 15

```
static long sequential (int n) {
```

 $O(n) + O(n) = O(n)$

```
    long c = 0;
```

 $n = 5$

```
    for (int i = 0; i < n; i++) c++;
```

 $i = 0, 1, 2, 3, 4$

```
    for (int i = 0; i < n; i++) c++;
```

 $i = 0, 1, 2, 3, 4$

```
    return c;
```

 $c = 10$

```
}
```

```
static long sqrtLoop (int n) {
```

 $O(\sqrt{n})$

```
    long c = 0;
```

```
    for (long i = 1; i * i <= n; i++) c++;
```

 $n = 10$

```
    return c;
```

 $i = 1, 2, 3, 4$

```
}
```

 $c = 0 + 1 + 1 + 1 = 3$
 $1, 4, 9, 16$

```
static long constant (int n) {
```

```
    long c = 0;
```

 $O(1)$

```
    for (int i = 0; i < 100; i++) c++;
```

```
    return c;
```

```
}
```

Recursion Cost: java

```
static long fib (int n) {
```

```
    calls ++;
```

```
    if (n < 2) return n;
```

```
    return fib (n-1) + fib (n-2);
```

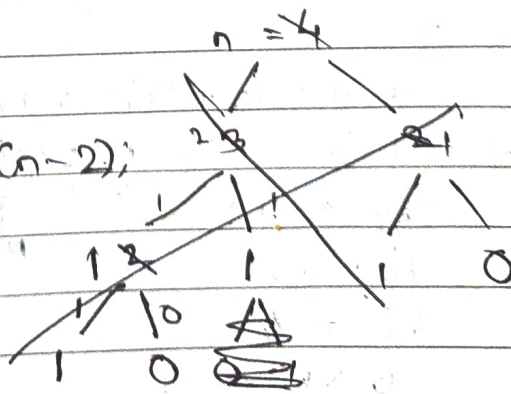
```
}
```

 $\text{fib}(4) = 3$

```
    fib(3)  fib(2)
```

```
    fib(2)  fib(1)  fib(1)  fib(0)
```

```
    fib(1)  fib(0)
```



Recursion Cost

public class RecursionCost {

static long calls;

// $T(n) = T(n-1) + c \rightarrow O(n)$ time, $O(n)$ stack space

static long factorial(int n) {

calls++;

if ($n \leq 1$) return 1;

return $n * \text{factorial}(n-1)$; $\rightarrow 24$

Call = 4

$n = 4$
/ \ 6

4 fact(3)

/ \ 2

3 fact(2)

2 / \ 1

2 fact(1)

1

// $T(n) = T(n/2) + c$

static int binarySearch(int[] a, int key, int lo, int hi) {

calls++;

if ($lo > hi$) return -1;

int mid = $lo + (hi - lo) / 2$; mid = 6

if ($a[mid] == key$) return mid;

if ($key < a[mid]$) return binarySearch(a, key, lo, mid-1);

return binarySearch(a, key, mid+1, hi);

}

mid = $lo + (hi - lo) / 2$; $\rightarrow 0 + (4 - 0) / 2 = 2$

int a[] = { 1, 2, 6, 7, 5 }

key = 7; = 6

lo = 0

hi = 4

lo > hi $\rightarrow -1$

(a[], key, 0, 1)

(a[], 7, 3, 4)

Mid

// $T(n) = T(n-1) + T(n-2) + C \rightarrow O(2^n)$ (exponential)

```
static long fib (int n) {
```

```
    calls++;
```

```
    if (n < 2) return n;
```

```
    return fib (n-1) + fib (n-2);
```

```
public static void main (String [] args) {
```

```
    for (int n : new int [] {10, 20}) {
```

```
        calls = 0; factorial(n);
```

```
        sout ("factorial(" + n + ") calls = " + calls);
```

```
int [] a = new int [1 < 20];     $2^{20} = 1,048,576$  sorted
```

```
for (int i = 0; i < a.length; i++) a[i] = i;
```

```
calls = 0; binarySearch(a, -1, 0, a.length-1);
```

```
Sout sout ("binarySearch + calls + " (log2 n = 20)");
```

```
for (int n = 10; n <= 30; n += 5) {
```

```
    calls = 0;
```

```
    long t = System.nanoTime();
```

```
    long v = fib(n);
```

```
    long ms = (System.nanoTime() - t) / 1000000;
```

```
    sout ("fib(%d) = %-8d ms calls = %d, 12d time = %d
```

```
ms %n",
```

```
        n, v, calls, ms);
```

LinearSearchCount.java

```
public class LinearSearchCount {
    static int comparisons; // global counter

    static int linearSearch(int[] a, int key) {
        comparisons = 0;
        for (int i = 0; i < a.length; i++) {
            comparisons++;
            if (a[i] == key) return i;
        }
        return -1;
    }
}
```

key = 3
a = 0 3 5 9
a[i] == key

```
public static void main (String [] args) {
    int n = 1000;
    int[] a = new int[n];
    for (int i = 0; i < n; i++) {
        a[i] = i * 3;
    }
    a[] = 0 3 6 9
```

```
LinearSearch(a, 0);
System.out.println("Best Case " + comparisons);
```

```
LinearSearch(a, a[n/2]);
System.out.println("Middle " + comparisons);
```

```
LinearSearch(a, a[n-1]);
System.out.println("Worst " + comparisons);

long total = 0;
for (int i = 0; i < n; i++) {
    LinearSearch(a, a[i]);
    total += comparisons;
}
System.out.println("Average " + (double) total / n);
```

String Concat Trap

```
public class StringConcatTrap {
```

```
    main (String []) {
```

```
        for (int n = 10000; n <= 80000; n *= 2) {
```

10000

20000

40000

80000

```
            long t = System.nanoTime();
```

```
            String s = "";
```

```
            for (int i = 0; i < n; i++) s += 'x'; // O(n^2)
```

```
            long slow = (System.nanoTime() - t) / 1000000;
```

```
            t = System.nanoTime();
```

```
            String Builder sb = new StringBuilder();
```

```
            for (int i = 0; i < n; i++) sb.append('x'); // O(n)
```

```
            String = sb.toString();
```

```
            long fast = (System.nanoTime() - t) / 1000000;
```

Program To Interface

→ List

```
static long sumByIndex (List<Integer> list) {
```

```
    long s = 0;
```

```
    for (int i = 0; i < list.size(); i++) s += list.get(i);
```

```
    return s;
```

```
}
```

1 2 0 1 2

```
static long sumByIterator (List<Integer> list) {
```

```
    long s = 0;
```

```
    Iterator<Integer> it = list.iterator();
```

```
    while (it.hasNext()) s += it.next();
```

```
    return s;
```

```
}
```

ArrayList

20000

```

static void run(String name, List<Integer> list, int n) {
    for (int i = 0; i < n; i++) list.add(i); // i = 0, 19999
    long t = System.nanoTime();
    long a = sumByIndex(list);
    long b = (System.nanoTime() - t);
    t = System.nanoTime();
    long c = sumByIterator(list);
    sout(" ");
}

```

main {

```

for (int n = 20000; n <= 80000; n *= 2) {
    run("ArrayList", new ArrayList<>(), n);
    run("LinkedList", new LinkedList<>(), n);
}

```

List Traps

main {

```

for (int n = 25000; n <= 200000; n *= 2) {
    List<Integer> list = new ArrayList<>();
    for (int i = 0; i < n; i++) list.add(i); // list = [25000, ...]
    long t = System.nanoTime();
    while (!list.isEmpty()) list.remove(0);
    long slow = (System.nanoTime() - t) / 1000000;
}

```

```

Deque<Integer> dq = new ArrayDeque<>();
for (int i = 0; i < n; i++) dq.add(i);
t = System.nanoTime();
while (!dq.isEmpty()) dq.poll();
long fast = (System.nanoTime() - t) / 1000000;
sout;

```